

Coda-C Library

MANPAGES

LIBRARY

coda-c
#include <coda-c.h>

NAME

Array_Join — combine Char type elements into a string

SYNOPSIS

Char
Array_Join(*Array array, char *delimiter*);

DESCRIPTION

Combine the string/Char type elements of an array to create a new string by inserting 'delimiter' between the elements. The length of Char elements is determined by strlen().

NAME

Array_NewBlock — create a new dynamic array of pointers

SYNOPSIS

Array
Array_NewBlock(*Array proto, int count, pointer block*);

DESCRIPTION

Create a new dynamic array from pointer block and retain objects if 'Array'. Prototypes: default/NULL is Array, CArray, etc.

NAME

Array_addObject — add item to the end of an array
Array_add — AKA

SYNOPSIS

Obj
Array_addObject(*Array self, Obj obj*);

DESCRIPTION

Equivalent to Array_insertAt() with index of Array_count().

NAME

Array_get_count — number of items in an array
Array_count — AKA

SYNOPSIS

int
Array_get_count(*Array self*);

DESCRIPTION

Return number of items in an array. Note: If (self) is NULL this function will return Zero.

NAME

Array_get_name — get the name property
Array_name — AKA

SYNOPSIS

Char
Array_get_name(*Array self*);

DESCRIPTION

Get the Char object property 'name' for an array.

NAME

Array_insertAt — insert item into array

SYNOPSIS

Obj
Array_insertAt(*Array self*, *int index*, *Obj obj*);

DESCRIPTION

Insert item into an existing array and retaining/keepO() the item. Note: The index should be valid or equal to the Array_count().

NAME

Array_insertBlock — insert block of items to array and retain them

SYNOPSIS

void
Array_insertBlock(*Array self*, *int index*, *int count*, *pointer block*);

DESCRIPTION

Insert items into an array, retaining the new items.

NAME

Array_rawAddress — get base address of array

SYNOPSIS

pointer
Array_rawAddress(*Array self*);

DESCRIPTION

Return the location of internal pointer data. Note: This address will become invalid if the Array is modified.

NAME

Array_removeAll — remove all item from an array

SYNOPSIS

```
void  
Array_removeAll(Array self);
```

DESCRIPTION

Remove all items from an array and releasing/freeO() them.

NAME

Array_removeAt — remove item from array

SYNOPSIS

```
void  
Array_removeAt(Array self, int dix);
```

DESCRIPTION

Remove an item from an array and releasing/freeO() the item.

NAME

Array_removeBlock — remove a block of items

SYNOPSIS

```
void  
Array_removeBlock(Array self, int dix, int count);
```

DESCRIPTION

Remove count items from an array.

NAME

Array_removeLast — remove the last item in an array

SYNOPSIS

```
void  
Array_removeLast(Array self);
```

DESCRIPTION

Equivalent to `Array_removeAt()` with index of `Array_count()-1`.

NAME

Array_removeObject — remove all items equal to object

SYNOPSIS

int
Array_removeObject(*Array self, Obj object*);

DESCRIPTION

Remove all occurrences of an item from an array. Note: Items removed must have the same address as the specified object.

RETURN VALUES

Returns the number items removed.

NAME

Array_replaceAt — replace an item in an array

SYNOPSIS

Obj
Array_replaceAt(*Array self, int ix, Obj obj*);

DESCRIPTION

Replace an item in an array, and for Array's of objects: freeO(old object), keepO(new object).

NAME

Array_set_name — set the name property

SYNOPSIS

void
Array_set_name(*Array self, char* name*);

DESCRIPTION

Set the Name of an array as a Char object copy of 'name'. Note: If (name) is NULL or an empty string the property is set to NULL.

NAME

Array_subInt — object at index

Array_sub — AKA

SYNOPSIS

Obj
Array_subInt(*Array self, int ix*);

DESCRIPTION

Return the object or pointer specified by index. Note: The index must be valid.

NAME

Array_takeBlock — insert block without retaining

SYNOPSIS

void
Array_takeBlock(*Array self, int index, int count, pointer block*);

DESCRIPTION

Insert items into an array, taking ownership of the new items.

NAME

Array_takeObject — add object and take ownership
Array_take — AKA

SYNOPSIS

void
Array_takeObject(*Array self, Obj \$CONSUMED obj*);

DESCRIPTION

Add an object to an array and without retaining.

EXAMPLES

Array_takeObject(list,Char_F("The value is %d.",45));

NAME

Char_F — new Char object printf style.

SYNOPSIS

Char \$FORMAT12
Char_F(*char *cs, . . .*);

DESCRIPTION

Return a new Char string like printf(). Note: 4K maximum string size.

NAME

Char_value — new Char object from string.

SYNOPSIS

Char
Char_value(*const char *string*);

DESCRIPTION

Return a new Char string object as copy of string.

NAME

Class_dictionary — storage for registered classes

SYNOPSIS

CDictionary

Class_dictionary(*void*);

DESCRIPTION

A raw C Dictionary of registered class objects.

NAME

Class_register — record class objects

SYNOPSIS

void

Class_register(*Obj obj*);

DESCRIPTION

Store a class object in the Class_dictionary(). Note: Called by CodaClass() macro.

NAME

Dictionary_AllKeys — create list of all keys

SYNOPSIS

Pointer

Dictionary_AllKeys(*Dictionary self*);

DESCRIPTION

Return a new Pointer object with pointers to the Dictionary's keys or NULL if the Dictionary is empty.

SEE ALSO

Dictionary_keywordFromWord(3)

NAME

Dictionary_get_count — number of items

Dictionary_count — AKA

SYNOPSIS

int

Dictionary_get_count(*Dictionary self*);

DESCRIPTION

Return the number of Keyword's in a Dictionary. Note: Will return 0 for dict==NULL.

NAME

Dictionary_get_name — get the name property

Dictionary_name — AKA

SYNOPSIS

```
Char
Dictionary_get_name(Dictionary self);
```

DESCRIPTION

Get the Char object property 'name' for a Dictionary.

NAME

Dictionary_keywordFromWord — MACRO: Keyword pointer from internal dictionary key

SYNOPSIS

```
Keyword
Dictionary_keywordFromWord(char *word);
```

DESCRIPTION

Returns a Keyword pointer. Warning: This function returns a raw pointer to data that has the same structure as the object Keyword. This raw pointer should only be used to retrieve the contents of a Dictionary. This function can only be called with a Keyword's word member returned from Dictionary_scan or Dictionary_next.

SEE ALSO

Dictionary_scan(3), Dictionary_next(3), Dictionary_AllKeys(3)

NAME

Dictionary_keywordSub — return a Keyword like pointer for an element

SYNOPSIS

```
Keyword
Dictionary_keywordSub(Dictionary dict, int index);
```

DESCRIPTION

Returns the Nth (starting with zero) element of a dictionary or NULL. Warning: This function returns a raw pointer to data that has the same structure as the object Keyword. These raw pointers should only be used to retrieve the contents of a Dictionary.

SEE ALSO

Dictionary_scan(3)

NAME

Dictionary_next — return the next element of a dictionary

SYNOPSIS

```
Keyword
Dictionary_next(Keyword element);
```

DESCRIPTION

Returns the next element of a dictionary or NULL. Warning: This function returns a raw pointer to data that has the same structure as the object Keyword. These raw pointers should only be used to retrieve the contents of a Dictionary. This function can only be called with a non-NULL Keyword

returned from Dictionary_scan or Dictionary_next.

SEE ALSO

Dictionary_scan(3)

NAME

Dictionary_removeKey — remove a key/item pair

SYNOPSIS

```
bool
Dictionary_removeKey(Dictionary self, char *key);
```

DESCRIPTION

Remove a Keyword from a dictionary and for object Dictionary's, freeO() the related item. Note: This call supports non object dictionaries such as CDictionary.

RETURN VALUES

Returns if the key was removed.

NAME

Dictionary_removeObject — remove all keys with item equal to object

SYNOPSIS

```
int
Dictionary_removeObject(Dictionary self, Obj obj);
```

DESCRIPTION

Remove all occurrences of an item from a dictionary, along with the related key. Note: Items removed must have the same address as the specified object.

RETURN VALUES

Returns the number items removed.

NAME

Dictionary_scan — return the first keyword in a dictionary

SYNOPSIS

```
Keyword
Dictionary_scan(Dictionary dict);
```

DESCRIPTION

Returns a pointer to the first keyword in a dictionary or NULL. Warning: This function returns a raw pointer to data that has the same structure as the object Keyword. These raw pointers should only be used to retrieve the contents of a Dictionary. Note: This function returns NULL for a NULL dictionary.

EXAMPLES

```
for(Keyword key=Dictionary_scan(dict);key;key=Dictionary_next(key)) {
    Msg_("Key=%s, item=%p",key->word,key->item);
}
```


SEE ALSO

Dictionary_next(3), Dictionary_keywordFromWord(3)

NAME

Dictionary_setKey — add Keyword to dictionary
Dict_set — AKA

SYNOPSIS

void
Dictionary_setKey(*Dictionary self, char *key, Obj obj*);

DESCRIPTION

Add a new key to a dictionary, by copying the key and storing the object. For Dictionary types the object will be retained with keepO(). Note: If called with a NULL object the key will be removed.

NAME

Dictionary_set_name — set the name property

SYNOPSIS

void
Dictionary_set_name(*Dictionary self, char* name*);

DESCRIPTION

Set the Name of a Dictionary as a Char object copy of 'name'. Note: If (name) is NULL or an empty string the property is set to NULL.

NAME

Dictionary_subKey — find object by key
Dict_sub — AKA

SYNOPSIS

Obj
Dictionary_subKey(*Dictionary self, char *key*);

DESCRIPTION

Return the item associated with key or NULL.

NAME

Dictionary_takeKey — set object and take ownership
Dict_take — AKA

SYNOPSIS

void
Dictionary_takeKey(*Dictionary self, char *key, Obj \$CONSUMED obj*);

DESCRIPTION

Set an new key/item pair into a Dictionary and reduce the object's retain count.

EXAMPLES

```
Dictionary_takeKey(dict,"value",Huge_Value(45));
```

SEE ALSO

Dictionary_setKey(3)

NAME

Die_ — MACRO: send a message adding __func__ and exit

SYNOPSIS

```
void  
Die_(const char* format, . . .);
```

DESCRIPTION

Create a printf style message string (garnished with 'function'), and call Die_Object(msg,1).

SEE ALSO

Quit_(3)

NAME

Die_Object — helper for Quit_() and Die_() // plist:lib

SYNOPSIS

```
Obj $NORETURN  
Die_Object(Char $CONSUMED msg, int trace);
```

DESCRIPTION

Process Char message and free it. Messages are sent by default to 'stderr' unless the 'Die_hook' function pointer is set. In either case the 'msg' is freed and exit(-1) is called terminating the program.

NAME

Global_objectAddress — record global objects

SYNOPSIS

```
void  
Global_objectAddress(pointer address);
```

DESCRIPTION

Record the location of global objects, which is useful for leak testing. Note: The address should be the address of a global object variable (&global_object).

NAME

Global_objectsZero — freeO and zero global objects

SYNOPSIS

void
Global_objectsZero(*void*);

DESCRIPTION

Free global objects. This is useful for leak testing.

NAME

Keyword_TakeItem — create Keyword, take ownership of item

SYNOPSIS

Keyword
Keyword_TakeItem(*char *word, Obj item*);

DESCRIPTION

Returns a new Keyword object that retains 'item' and copies 'word' then decreases the retain count.

NAME

Keyword_Value — create a Keyword object

SYNOPSIS

Keyword
Keyword_Value(*char *word, Obj item*);

DESCRIPTION

Returns a new Keyword object that retains 'item' and copies 'word'.

NAME

Keyword_get_item — same as self->item

SYNOPSIS

Obj
Keyword_get_item(*Keyword self*);

DESCRIPTION

Returns the Keyword's item object. This is provided to avoid extra casting to access the item.

NAME

Keyword_set_item — replace a Keyword's item/object

SYNOPSIS

void
Keyword_set_item(*Keyword self, Obj object*);

DESCRIPTION

Replace a Keyword's item by freeO(old) and keepO(new). HELPER.

NAME

Log_ — MACRO: send a message, normally to a log file

SYNOPSIS

void
Log_(*const char* format, . . .*);

DESCRIPTION

Create a printf style message string and call **Msg_Object**(msg,1).

SEE ALSO

Msg_(3)

NAME

Memory_alloc0 — allocate a classless Object
alloc0 — AKA

SYNOPSIS

Obj
Memory_alloc0(*int size*);

DESCRIPTION

Allocate a Void class Object with zeroed contents.

NAME

Memory_class — **classO()**
class0 — AKA

SYNOPSIS

*CodaCLASS**
Memory_class(*Obj obj*);

DESCRIPTION

get object's class pointer or **&Isa_Void**. aka **classO()**

NAME

Memory_clean — attribute 'cleanO' helper function

SYNOPSIS

void
Memory_clean(*const void *vp*);

DESCRIPTION

This function is used to **freeO()** Obj variables with attribute 'cleanO' when they go out of scope.

SEE ALSO

`cleanO(3)`

NAME

Memory_count — get retain count
count0 — AKA

SYNOPSIS

int4
Memory_count(*Obj obj*);

DESCRIPTION

Return the retain count for an Object.

NAME

Memory_free — reduce retain count
free0 — AKA

SYNOPSIS

void
Memory_free(*Obj obj*);

DESCRIPTION

Reduce the retain count of an Object and if zero destroy it.

NAME

Memory_keep — increase retain count by 1
keep0 — AKA

SYNOPSIS

void
Memory_keep(*Obj obj*);

DESCRIPTION

Increase the retain count of an Object.

NAME

Memory_kind — get class name
kind0 — AKA

SYNOPSIS

*char**
Memory_kind(*Obj obj*);

DESCRIPTION

Return the object's class string.

NAME

Memory_new0 — Create a new object with count

SYNOPSIS

Obj
Memory_new0(*Obj template, int count*);

DESCRIPTION

Returns a new arrayed object of the same class as the template containing 'count' elements or with 'count' additional bytes for 'kize' object classes. The template is often a class object.

SEE ALSO

newO(3), newOC(3)

NAME

Memory_normalize — get standard Object address
normal0 — AKA

SYNOPSIS

Obj
Memory_normalize(*Obj obj*);

DESCRIPTION

Return the normal address of an object by converting a spindle address. Spindle addresses are used for subclassed object segments.

NAME

Memory_size — get an Object's size
size0 — AKA

SYNOPSIS

int4
Memory_size(*Obj obj*);

DESCRIPTION

Return the size of an Object or segment of an Object.

NAME

Memory_spindle — get a spindle address
spindle0 — AKA

SYNOPSIS

Obj
Memory_spindle(*Obj obj, CodaCLASS* newClass*);

DESCRIPTION

Return the spindle address of an object that matches the provided class or NULL. The class is normally of the form: &lsa_CLASS.

NAME

Memory_superObject — get the next segment address
super0 — AKA

SYNOPSIS

Obj
Memory_superObject(*Obj obj*);

DESCRIPTION

Return the next spindle address of a subclassed object or NULL.

NAME

Msg_ — MACRO: send a message, normally to the terminal / stderr

SYNOPSIS

void
Msg_(*const char* format, . . .*);

DESCRIPTION

Create a printf style message string and call **Msg_Object**(msg,0).

SEE ALSO

Log_(3)

NAME

Msg_Object — helper for **Msg_()** and **Log_()** // plist:lib

SYNOPSIS

void
Msg_Object(*Char \$CONSUMED msg, int log*);

DESCRIPTION

Process Char message and free it. Messages are sent by default to 'stderr' unless the 'Msg_hook' function pointer is set. In either case the 'msg' is freed.

NAME

OHasSuper — MACRO: does object have a virtual superclass function for SIG?

SYNOPSIS

*void**
OHasSuper(*Obj obj, token SIG*);

DESCRIPTION

Does object have a virtual superclass function for SIG? Note: 'class' must be defined to use this function, like during class creation.

SEE ALSO

ORespond3(3)

NAME

ORespond3 — function address & object

SYNOPSIS

```
void *
ORespond3(Obj obj, char *sel, Obj *spindle, int noDie, CodaCLASS *superOf);
```

DESCRIPTION

Return multiplexed function address and spindle. Supports super_(), OHasSuper(), etc.

NAME

OResponds — MACRO: does object have a virtual function for SIG?

SYNOPSIS

```
void*
OResponds(Obj obj, token SIG);
```

DESCRIPTION

Does object have a virtual function for SIG?

SEE ALSO

ORespond3(3)

NAME

OisaClass — does object inherit from class

SYNOPSIS

```
bool
OisaClass(Obj obj, Obj classobj);
```

DESCRIPTION

Supports isa_() macro.

NAME

Os — MACRO: generate a string constant object

SYNOPSIS

```
ConstChar
Os(string constant);
```


DESCRIPTION

Returns a static string constant object impervious to retain/release. Note: Because these objects are quickly generated by a C compiler, they return a the C string length + 1 for sizeO().

NAME

Pointer_NewBlock — block allocator

SYNOPSIS

Pointer

Pointer_NewBlock(*Pointer self, int count, pointer block*);

DESCRIPTION

Create a new Pointer object from block of pointers with count items. Note: proto is ignored.

NAME

Pointer_Value — create a Pointer object

SYNOPSIS

Pointer

Pointer_Value(*pointer value*);

DESCRIPTION

Returns a new Pointer object with a single value.

NAME

Pointer_length — return the number of pointers in a NULL terminated list

SYNOPSIS

int

Pointer_length(*pointer address*);

DESCRIPTION

Return the number of pointers in a NULL terminated block of pointers. Note: If the input address is NULL this function will return Zero.

EXAMPLES

```
char *list[]={ "apple", "grape", "orange", 0, "foo", 0};
printf("# of pointers: %d\n", Pointer_length(list)); // 3
```

NAME

Quit_ — MACRO: send a message, normally to the terminal and exit

SYNOPSIS

void

Quit_(*const char* format, . . .*);

DESCRIPTION

Create a printf style message string and call `Die_Object(msg,0)`.

SEE ALSO

`Die_(3)`

NAME

Root_get_count — return logical count of elements

Pointer_count — AKA

Root_count — AKA

SYNOPSIS

int

Root_get_count(*Root self*);

DESCRIPTION

Returns the logical item count, or additional bytes for (kize) objects.

NAME

Signature_dictionary — storage for unique signature strings

SYNOPSIS

CDictionary

Signature_dictionary(*void*);

DESCRIPTION

A raw C Dictionary of registered signatures.

NAME

Signature_fromString — lookup unique signature constants

SYNOPSIS

Char

Signature_fromString(*char *str*);

DESCRIPTION

Find signature string-constant by name.

NAME

Signature_register — record unique signature string constants

SYNOPSIS

void

Signature_register(*Char sig*);

DESCRIPTION

Store signature string-constants in the `Signature_dictionary()`. These constants are used to find virtual functions for the function multiplexor.

NAME

ToContainer — generate diagnostic values for objects, tools, mux

SYNOPSIS

Char

ToContainer(*Obj container*);

DESCRIPTION

A diagnostic function that returns a string representation of its value. Note: These strings can get very long.

NAME

cleanC — MACRO: attribute for pointer variables

SYNOPSIS

cleanC pointer NAME=

cleanC(*IGNORE_THIS_FAKE_FUNCTION*);

DESCRIPTION

Attribute for pointer variables that causes `freeC()` to be called when they go out of scope. It also makes these variables 'const' so they are not changed before `freeC()`.

EXAMPLES

`cleanC pointer name=allocC(64); // name will be freed automatically.`

SEE ALSO

`clean_C(3)`

NAME

cleanO — MACRO: attribute for object variables

SYNOPSIS

cleanO Obj NAME=

cleanO(*IGNORE_THIS_FAKE_FUNCTION*);

DESCRIPTION

Attribute for object variables that causes `freeO()` to be called when they go out of scope. It also makes these variables 'const' so they are not changed before `freeO()`.

EXAMPLES

`cleanO Array list=newO(Array); // 'list' will be freed automatically.`

SEE ALSO

Memory_clean(3)

NAME

clean_C — attribute 'cleanC' helper function

SYNOPSIS

```
void
clean_C(const void *vp);
```

DESCRIPTION

This function is used to freeC() pointer variables with attribute 'cleanC' when they go out of scope.

NAME

codac_sig — register virtual function by class and signature

SYNOPSIS

```
void
codac_sig(pointer cmux, char *key, pointer data);
```

DESCRIPTION

Register functions with **codac_sig**(Class_Object,Signature,function pointer). Note: This is normally call with a macro like CodaSig().

NAME

cs_blockCmp — MACRO: compare blocks of memory

SYNOPSIS

```
int
cs_blockCmp(pointer s1, pointer s2, int len);
```

DESCRIPTION

Compare two blocks of memory for sorting. Returns -1, 0, or 1.

NAME

cs_blockCopy — MACRO: copy a block of memory

SYNOPSIS

```
void
cs_blockCopy(pointer dest, pointer source, int nel);
```

DESCRIPTION

Copy a block of memory. Note: The source and destination should not overlap.

SEE ALSO

memcpy(3)

NAME

cs_blockFill — MACRO: fill a block of memory with a character

SYNOPSIS

```
void  
cs_blockFill(pointer dest, int cc, int len);
```

DESCRIPTION

Fill a block of memory with a character.

SEE ALSO

memset(3)

NAME

cs_blockZero — MACRO: zero a block of memory

SYNOPSIS

```
void  
cs_blockZero(pointer dest, int len);
```

DESCRIPTION

Zero a block of memory.

SEE ALSO

memset(3), bzero(3)

NAME

cs_exact — MACRO: exact compare strings

SYNOPSIS

```
bool  
cs_exact(char* str1, char* str2);
```

DESCRIPTION

Returns true if the two strings are the same.

NAME

cs_length — MACRO: length of a string

SYNOPSIS

```
int  
cs_length(char* str);
```

DESCRIPTION

Returns the number of characters before encountering a NUL character (0).

SEE ALSO

strlen(3)

NAME

cs_strcopy — MACRO: copy a string

SYNOPSIS

*char**
cs_strcopy(*char* dest, char* source*);

DESCRIPTION

Copy non-overlapping string, returning the destination. Aka for strcpy().

SEE ALSO

strcpy(3)

NAME

freeC — deallocates memory

SYNOPSIS

void
freeC(*pointer address*);

DESCRIPTION

Deallocates memory by calling free(). Null addresses are ignored.

SEE ALSO

free(3), allocC(3)

NAME

isa_ — MACRO: does object inherit from 'class'

SYNOPSIS

bool
isa_(*Obj obj, token class*);

DESCRIPTION

Example: if (isa_(thing,Array)) { ... }

SEE ALSO

OisaClass(3)

NAME

new0 — MACRO: Create a new object

SYNOPSIS

```
Obj
new0(token class);
```

DESCRIPTION

Returns a new object of 'class'.

SEE ALSO

newOC(3)

NAME

newOC — MACRO: Create a new object with count

SYNOPSIS

```
Obj
newOC(token class, int count);
```

DESCRIPTION

Returns a new arrayed object of 'class' containing 'count' elements or with 'count' additional bytes for 'kize' object classes.

SEE ALSO

newO(3)

NAME

obj_ — MACRO: the Coda-C Objective call

SYNOPSIS

```
Various*
obj_(token SIG, Obj obj, . . .);
```

DESCRIPTION

The Coda-C Objective call where the return value and arguments are defined by the Signature. The virtual function is looked up from the object's class and signature key sig_SIG.

SEE ALSO

super_(3)

NAME

set0 — MACRO: variable=object assignment with retain/release coding

SYNOPSIS

```
Obj
set0(token variable, token object);
```

DESCRIPTION

variable=object with retain/release. Note: This macro preserves type checking on the assignment and does keepO first to prevent deallocation if assigning same value! Performs: keepO(object),freeO(variable),variable=object.

SEE ALSO

setO_take(3)

NAME

set0_take — MACRO: variable=object assignment taking object ownership

SYNOPSIS

Obj
set0_take(*token variable*, *token object*);

DESCRIPTION

variable=object with release. Performs: freeO(variable),variable=object.

SEE ALSO

setO(3)

NAME

sizeat — MACRO: take sizeof known pointer->object type

SYNOPSIS

size_t
sizeat(*token type*);

DESCRIPTION

Returns sizeof() a pointer type's object. Note: Computed at compile time.

SEE ALSO

sizeof(3)

NAME

super_ — MACRO: the Coda-C Objective super class call

SYNOPSIS

Various
super_(*token SIG*, *Obj obj*, . . .);

DESCRIPTION

The Coda-C Objective super class call is just like the objective call `obj_` except it may only be used where 'class' has been defined, like during class creation.

SEE ALSO

obj_(3)